

SQL Views, View Updatability and Recursive CTE

Aim

To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive Common Table Expression (CTE) to display employee reporting chains.

Software / Requirements

- MySQL 8.0 or later
- MySQL Workbench
- CompanyDB database created in Experiments 1 and 2

Theory

A view is a virtual table based on a SELECT query. Views can simplify complex queries and provide controlled access to underlying data. A simple single-table view may be updatable, subject to DBMS rules. Views containing GROUP BY or aggregate functions are not directly updatable because their rows represent groups rather than individual base-table rows. A recursive CTE contains an anchor member and a recursive member and is useful for hierarchical data such as employee-manager relationships.

1. Extend the Employee Schema for Reporting Hierarchy

Add a Manager_ID column that references Emp_ID in the same Employee table.

```
USE CompanyDB;

ALTER TABLE Employee
ADD COLUMN Manager_ID INT NULL;

ALTER TABLE Employee
ADD CONSTRAINT fk_employee_manager
FOREIGN KEY (Manager_ID) REFERENCES Employee(Emp_ID);
```

Assign sample reporting relationships:

```
-- Artificial Intelligence
UPDATE Employee SET Manager_ID = NULL WHERE Emp_ID = 101;
UPDATE Employee SET Manager_ID = 101 WHERE Emp_ID IN (102, 103);
UPDATE Employee SET Manager_ID = 102 WHERE Emp_ID IN (104, 105);
UPDATE Employee SET Manager_ID = 103 WHERE Emp_ID = 106;

-- Software Development
UPDATE Employee SET Manager_ID = NULL WHERE Emp_ID = 110;
UPDATE Employee SET Manager_ID = 110 WHERE Emp_ID IN (107, 108);
UPDATE Employee SET Manager_ID = 107 WHERE Emp_ID IN (109, 111);
UPDATE Employee SET Manager_ID = 108 WHERE Emp_ID = 112;

-- Cyber Security
UPDATE Employee SET Manager_ID = NULL WHERE Emp_ID = 115;
UPDATE Employee SET Manager_ID = 115 WHERE Emp_ID IN (113, 114);
UPDATE Employee SET Manager_ID = 113 WHERE Emp_ID IN (117, 118);
UPDATE Employee SET Manager_ID = 114 WHERE Emp_ID = 116;

-- Data Science
UPDATE Employee SET Manager_ID = NULL WHERE Emp_ID = 119;
UPDATE Employee SET Manager_ID = 119 WHERE Emp_ID IN (120, 121);
```

```

UPDATE Employee SET Manager_ID = 120 WHERE Emp_ID IN (122, 123);
UPDATE Employee SET Manager_ID = 121 WHERE Emp_ID = 124;

-- Human Resources
UPDATE Employee SET Manager_ID = NULL WHERE Emp_ID = 125;
UPDATE Employee SET Manager_ID = 125 WHERE Emp_ID IN (126, 129);
UPDATE Employee SET Manager_ID = 126 WHERE Emp_ID IN (127, 128, 130);

```

2. Create Department Salary Summary View

```

CREATE OR REPLACE VIEW Department_Salary_Summary AS
SELECT d.Dept_ID,
       d.Dept_Name,
       COUNT(e.Emp_ID) AS Employee_Count,
       SUM(e.Salary) AS Total_Salary,
       AVG(e.Salary) AS Average_Salary,
       MIN(e.Salary) AS Minimum_Salary,
       MAX(e.Salary) AS Maximum_Salary
FROM Department d
LEFT JOIN Employee e ON d.Dept_ID = e.Dept_ID
GROUP BY d.Dept_ID, d.Dept_Name;

```

Display the view:

```

SELECT * FROM Department_Salary_Summary;

```

Observation: this view contains GROUP BY and aggregate functions, so it is not directly updatable.

3. Create Employee Hierarchy View

```

CREATE OR REPLACE VIEW Employee_Hierarchy AS
SELECT e.Emp_ID,
       e.Emp_Name AS Employee_Name,
       e.Job_Title,
       e.Dept_ID,
       d.Dept_Name,
       e.Manager_ID,
       m.Emp_Name AS Manager_Name
FROM Employee e
LEFT JOIN Department d ON e.Dept_ID = d.Dept_ID
LEFT JOIN Employee m ON e.Manager_ID = m.Emp_ID;

```

Display the hierarchy view:

```

SELECT *
FROM Employee_Hierarchy
ORDER BY Dept_ID, Manager_ID, Emp_ID;

```

4. Test Updatability of a Simple View

```

CREATE OR REPLACE VIEW Employee_Basic AS
SELECT Emp_ID, Emp_Name, Job_Title, Salary, Dept_ID
FROM Employee;

```

Test an update through the simple view:

```
UPDATE Employee_Basic
SET Salary = 86000
WHERE Emp_ID = 101;
```

Verify the base table:

```
SELECT Emp_ID, Emp_Name, Salary
FROM Employee
WHERE Emp_ID = 101;
```

Expected observation: the update succeeds because Employee_Basic is a simple projection of one base table.

5. Test Updatability of the Aggregate View

```
UPDATE Department_Salary_Summary
SET Average_Salary = 90000
WHERE Dept_ID = 1;
```

Expected observation: MySQL rejects the update because the view contains GROUP BY and aggregate functions.

6. Test Updatability of the Employee Hierarchy View

```
UPDATE Employee_Hierarchy
SET Manager_Name = 'Priya Verma'
WHERE Emp_ID = 103;
```

Expected observation: the update is rejected because Manager_Name is derived from a joined table and the view is not directly updatable as a whole.

7. Inspect View Definitions

```
SHOW CREATE VIEW Department_Salary_Summary;
SHOW CREATE VIEW Employee_Hierarchy;
SHOW CREATE VIEW Employee_Basic;
```

8. Recursive CTE to Display Reporting Chains

The anchor member selects top-level employees with no manager. The recursive member repeatedly finds employees whose Manager_ID matches an employee already found.

```
WITH RECURSIVE EmployeeChain AS (
  SELECT
    e.Emp_ID,
    e.Emp_Name,
    e.Manager_ID,
    0 AS Level,
    CAST(e.Emp_Name AS CHAR(1000)) AS Reporting_Chain
  FROM Employee e
  WHERE e.Manager_ID IS NULL
```

```

UNION ALL

SELECT
    e.Emp_ID,
    e.Emp_Name,
    e.Manager_ID,
    ec.Level + 1,
    CONCAT(ec.Reporting_Chain, ' -> ', e.Emp_Name)
FROM Employee e
INNER JOIN EmployeeChain ec
    ON e.Manager_ID = ec.Emp_ID
)
SELECT Emp_ID, Emp_Name, Manager_ID, Level, Reporting_Chain
FROM EmployeeChain
ORDER BY Reporting_Chain;

```

9. Recursive CTE for One Reporting Tree

```

WITH RECURSIVE ReportingTree AS (
    SELECT Emp_ID, Emp_Name, Manager_ID, 0 AS Level
    FROM Employee
    WHERE Emp_ID = 101

    UNION ALL

    SELECT e.Emp_ID, e.Emp_Name, e.Manager_ID, rt.Level + 1
    FROM Employee e
    INNER JOIN ReportingTree rt
        ON e.Manager_ID = rt.Emp_ID
)
SELECT Emp_ID, Emp_Name, Manager_ID, Level
FROM ReportingTree
ORDER BY Level, Emp_ID;

```

10. Recursive CTE with Hierarchy Indentation

```

WITH RECURSIVE EmployeeTree AS (
    SELECT Emp_ID, Emp_Name, Manager_ID, 0 AS Level
    FROM Employee
    WHERE Manager_ID IS NULL

    UNION ALL

    SELECT e.Emp_ID, e.Emp_Name, e.Manager_ID, et.Level + 1
    FROM Employee e
    INNER JOIN EmployeeTree et
        ON e.Manager_ID = et.Emp_ID
)
SELECT Emp_ID,
    CONCAT(REPEAT('    ', Level), Emp_Name) AS Employee_Hierarchy,
    Level
FROM EmployeeTree
ORDER BY Level, Emp_ID;

```

11. Verify Reporting Relationships

```
SELECT e.Emp_Name AS Employee,
       m.Emp_Name AS Manager
FROM Employee e
LEFT JOIN Employee m ON e.Manager_ID = m.Emp_ID
ORDER BY e.Emp_ID;
```

Output:

Dept_ID	Dept_Name	Employee_Count	Total_Salary	Average_Salary	Minimum_Salary	Maximum_Salary
1	Artificial Intelligence	6	520000.00	86666.666667	72000.00	105000.00
2	Software Development	6	465000.00	77500.000000	65000.00	95000.00
3	Cyber Security	6	516000.00	86000.000000	68000.00	98000.00
4	Data Science	6	506000.00	84333.333333	72000.00	100000.00
5	Human Resources	6	403000.00	67166.666667	58000.00	85000.00

Emp_ID	Employee_Name	Job_Title	Dept_ID	Dept_Name	Manager_ID	Manager_Name
101	Aarav Sharma	AI Engineer	1	Artificial Intelligence	NULL	NULL
102	Priya Verma	ML Engineer	1	Artificial Intelligence	101	Aarav Sharma
103	Rohan Gupta	AI Developer	1	Artificial Intelligence	101	Aarav Sharma
104	Ananya Singh	Research Engineer	1	Artificial Intelligence	102	Priya Verma
105	Karan Mehta	ML Engineer	1	Artificial Intelligence	102	Priya Verma
106	Neha Kapoor	AI Analyst	1	Artificial Intelligence	103	Rohan Gupta
110	Simran Kaur	Full Stack Developer	2	Software Development	NULL	NULL
109	Aditya Jain	Frontend Developer	2	Software Development	107	Rahul Kumar

120	Kriti Sharma	Data Analyst	4	Data Science	119	Akash Singh
121	Mohit Agarwal	Data Scientist	4	Data Science	119	Akash Singh
122	Riya Das	Data Analyst	4	Data Science	120	Kriti Sharma
123	Nikhil Bansal	ML Data Scientist	4	Data Science	120	Kriti Sharma
124	Sakshi Roy	BI Analyst	4	Data Science	121	Mohit Agarwal
125	Manish Verma	HR Manager	5	Human Resources	NULL	NULL
126	Divya Mehta	HR Executive	5	Human Resources	125	Manish Verma
129	Varun Gupta	Training Manager	5	Human Resources	125	Manish Verma
127	Suresh Kumar	Recruiter	5	Human Resources	126	Divya Mehta
128	Ayesha Khan	HR Executive	5	Human Resources	126	Divya Mehta
130	Nandini Sharma	HR Analyst	5	Human Resources	126	Divya Mehta

Emp_ID	Emp_Name	Salary
101	Aarav Sharma	86000.00

ERROR 1288 (HY000) at line 201: The target table Department_Salary_Summary of the UPDATE is not updatable

12. View Updatability Comparison

View	Main Feature	Directly Updatable?	Reason
Employee_Basic	Single base table	Yes*	Simple projection of Employee columns.

Department_Salary_Summary	GROUP BY + aggregates	No	Rows represent grouped/aggregated data.
Employee_Hierarchy	Multiple-table JOINS	No	Contains derived columns from joined tables.

* Actual behavior should be verified in the installed MySQL environment; view updatability depends on the view definition and DBMS rules.

13. Important Observations

- A view stores a query definition rather than a separate copy of the underlying data.
- Aggregate and GROUP BY views represent grouped results and are not directly updatable.
- A simple single-table projection can be updatable when it satisfies the DBMS rules.
- A recursive CTE consists of an anchor member and a recursive member.
- Manager_ID creates a self-referencing foreign-key relationship in Employee.
- The recursive CTE continues until no additional matching employees are found.

14. Result

SQL views for department salary summary and employee hierarchy were successfully created. View updatability was tested using a simple view and non-updatable aggregate/join views. A recursive CTE was implemented to traverse employee-manager relationships and display reporting chains with hierarchy levels.